



Centro Académico de Limón

Compiladores e Intérpretes

IC-5701

Grupo 60

Tarea 1

Gramática BNF de un lenguaje en paradigma imperativo

Profesor:

Allan Rodríguez Dávila

Estudiantes:

Alice Arias Salazar | 2023104639

Heldyis Agüero Espinoza |

Fecha de Entrega: 11/09/2026

II Semestre

2026

1 Descripción del problema

El presente trabajo nos coloca en un contexto actual donde la industria de fabricación de y configuración de circuitos integrados mantiene un crecimiento sostenido, impulsado por la creciente demanda de dispositivos electrónicos inteligentes, sensores IoT y demás hardware especializado. Estas necesidades en el desarrollo tecnológico traen consigo la necesidad de contar con herramientas de software cada vez más eficientes, ligeras y específicas, que permitan a los desarrolladores interactuar directamente con el hardware.

Teniendo esto en cuenta, la actividad plantea el escenario hipotético en el que existe un grupo de trabajadores que requiere crear un nuevo lenguaje de tipo imperado. El lenguaje busca ser compacto, que permita expresar de manera clara y directa las operaciones fundamentales que un sistema embebido requiere como lo son la declaración y manipulación de variables, estructuras de control de flujo, operaciones de diferentes tipos y clases y demás operaciones incluyendo el punto de entrada único de ejecución.

Para empezar a construir este lenguaje lo primero es definir su sintaxis, es decir, las reglas que determinan qué construcciones de código son válidas dentro del lenguajes y cuáles no. Esta definición formal constituye la base sobre la cual, en etapas posteriores del curso, se construirán el analizador léxico, el analizador sintáctico y, eventualmente, el analizador semántico del lenguaje.

Por lo tanto, el problema a resolver en esta primera tarea consiste en diseñar y formalizar una gramática libre de contexto, en notación BNF (Backus-Naur Form), capaz de generar el lenguaje imperativo descrito por los desarrolladores, cumpliendo con un conjunto específico de requerimientos funcionales y sintácticos, entre los que destacan:

- La definición de un símbolo inicial único y un conjunto bien definido de terminales y no terminales.
- El manejo de variables globales y locales, con tipado explícito y fuerte, sobre tipos primitivos (entero, flotante, booleano, carácter, cadena) y arreglos estáticos unidimensionales de tipo entero y flotante.
- La creación de sentencias de declaración (usando la palabra reservada `val`), de asignación (mediante el símbolo `=`) y de expresión pura (sin asignación).

- La construcción de expresiones que combinen literales, variables, arreglos y llamadas a funciones, respetando una jerarquía de precedencia de operadores claramente establecida (potencia, suma/resta, multiplicación/división/división entera, módulo).
- La incorporación de operadores aritméticos binarios y unarios, operadores relacionales (con la restricción de uso obligatorio de corchetes $\mathbb{J}$: $\mathbb{J}$ sobre sus operandos) y operadores lógicos (con la restricción de uso obligatorio de paréntesis ϵ : ϵ), todos con reglas sintácticas propias que impiden combinaciones arbitrarias entre distintos tipos de expresión.
- El soporte de estructuras de control típicas del paradigma imperativo: if-elif-else, while y for, así como las sentencias de control de flujo return y break.
- Funciones predefinidas de entrada y salida estándar (read y write) restringidas a literales y variables de tipos permitidos.
- La definición y el uso de funciones con parámetros tipados y valores de retorno, así como de un procedimiento principal único (main) de tipo void sin parámetros.
- El soporte de comentarios de una línea y de múltiples líneas.

Un desafío interesante de este problema, que también es una restricción descrita en el enunciado, es que la gramática no puede recurrir a producciones genéricas y ambiguas que combinen cualquier tipo de expresión bajo una regla recursiva. En su lugar, la gramática debe estatificarse a nivel jerárquico de expresión que refleje de forma correcta la precedencia, asociatividad y restricciones ambigüedades y que garantice generar construcciones sintácticamente válidas.

En resumen, el problema que esta tarea plantea resolver es el diseño de una especificación gramatical formal, no ambigua y jerárquicamente estructurada junto con los requerimientos funcionales establecidos, que sirva como fundamento teórico y práctico para el desarrollo posterior de las fases de análisis léxico y sintáctico de este nuevo lenguaje imperativo orientado a sistemas embebidos.

2 Diseño del programa

2.1 Terminales

```
3 ----- Terminales -----
4
5 Palabras reservadas
6
7 Variables, funciones y tipos de datos
8 "val"      (Creacion de variables)
9 "defun"    (Creacion de funcion)
10 "int"
11 "float"
12 "bool"
13 "char"
14 "string"
15 "void"
16 "true"
17 "false"
18 "dg"      (Variable global)
19 "dl"      (Variable local)
20 Estructuras condicionales
21 "if"
22 "elif"
23 "else"
24 Ciclos y control de funciones
25 "while"
26 "for"
27 "return"
28 "break"
29 Entrada y salida de datos
30 "read"    (Lectura o entrada de datos)
31 "write"   (Escritura o salida de datos)
32
33 Operadores aritmeticos
34 Binarios
35 "+"
36 "-"
37 "*"      (Multiplicacion)
38 "/"      (Division)
39 "/"      (Division entera)
40 "mod"    (Modulo)
41 "pot"    (Potencia)
42
43 Unarios
44 "-"      (Negacion)
45 "++"     (Incremento)
46 "--"     (Decremento)
47 Operadores relacionales
48 "<"
49 ">"
50 "<="
51 ">="
52 "=="
53 Operadores logicos
54 "&"      (and)
55 "||"     (or)
56 "!"      (not)
57 Delimitadores y simbolos literales
58 Bloques
59 "{"      (Abre )
60 "}"      (Cierra )
61 Parentesis
62 "("      (Abre )
63 ")"      (Cierra )
64 Indices de arreglos
65 "["      (Inicia )
66 "]"      (Cierra )
67 Otros
68 "~"      (Final de expresion)
69 "="      (Asignacion)
70 ";"      (Separador para parametros de funcion)
71 "."      (Punto decimal)
72 Comentarios
73 "/*"     (Comentario de linea)
74 "/*"     (Abre comentario multilinea)
75 "*/"     (Cierra comentario multilinea)
76
77 Digos
78 "0" "1" "2" "3" "4" "5" "6" "7" "8" "9"
79 Letras
80 "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s" "t" "u" "v" "w" "x" "y" "z" "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S" "T" "U" "V" "W" "X" "Y" "Z"
```

El conjunto de terminales de la gramática se organizó por categorías temáticas: palabras reservadas, operadores, delimitadores y símbolos literales. Además de los alfabetos base de dígitos y letras, esto con el fin de facilitar la lectura y mantenimiento.

Dentro de las palabras reservadas se agrupan los términos vinculados a la declaración de variables y funciones, tipos de datos, estructuras de control, ciclos y operaciones de entrada/salida. Todas las palabras reservadas por el lenguaje y no pueden emplearse como identificadores de variables o funciones. Es importante destacar el uso de los prefijos *dg* y *dl* para distinguir explícitamente entre variables de alcance global y local respectivamente.

En cuanto a los operadores aritméticos, se optó por representar la división entera, el módulo y la potencia mediante los símbolos *//*, *mod* y *pot*, en lugar de emplear una única barra o el símbolo %, con el objetivo de hacer más explícita su semántica. El símbolo *-* se reutiliza tanto como operador binario de resta como operador unario de negación.

Un rasgo distintivo del lenguaje es el uso de símbolos no convencionales como λ , θ y Σ como operadores lógicos o ζ , $?$, ϵ , $?$, ϵ , $?$, ϵ y $?$ como delimitadores de bloques, paréntesis e índices de arreglos, cumpliendo la función que en cumplen en otros lenguajes símbolos como *()*, *[]* y *{}*.

Finalmente, los terminales de dígitos y letras constituyen el alfabeto base a partir del cual se construyen los identificadores, números y cadenas de texto en las producciones correspondientes.

2.2 No terminales

documentos / no terminales	documentos / no terminales	documentos / no terminales
79 letras	138 Palabras reservadas y operadores	204 Comentarios
81 <i>No Terminales</i>	144 <i>palabraif</i>	205 <i>comentario_linea</i>
82 <i>Caracteres y texto</i>	145 <i>palabraelif</i>	206 <i>comentario_multilinea</i>
83 <i>Caracteres y texto</i>	146 <i>palabradef</i>	207 <i>comentario</i>
84 <i>Caracteres y texto</i>	147 <i>palabradef</i>	208 <i>Operaciones aritméticas</i>
85 <i>Caracteres y texto</i>	148 <i>palabrapot</i>	209 <i>valor_aritmetico</i>
86 <i>Caracteres y texto</i>	149 <i>operadorPotencia</i>	210 <i>expresion_potencia</i>
87 <i>Caracteres y texto</i>	150 <i>modulo</i>	211 <i>expresion_soma</i>
88 <i>Caracteres y texto</i>	151 <i>mod</i>	212 <i>expresion_multiplicacion</i>
89 <i>Caracteres y texto</i>	152 <i>clear</i>	213 <i>expresion_modulo</i>
90 <i>Caracteres y texto</i>	153 <i>describir</i>	214 <i>expresion_aritmetica</i>
91 <i>Caracteres y texto</i>	154 <i>varGlobal</i>	215 <i>operador_soma</i>
92 <i>Caracteres y texto</i>	155 <i>varLocal</i>	216 <i>operador_multiplicacion</i>
93 <i>Caracteres y texto</i>	156 <i>varcio</i>	217 <i>Operaciones relacionales</i>
94 <i>Caracteres y texto</i>	157 <i>Operadores</i>	218 <i>operadorRelacional</i>
95 <i>Caracteres y texto</i>	158 <i>simboloComentarioLinea</i>	219 <i>operando_relacional</i>
96 <i>Caracteres y texto</i>	159 <i>simboloInicioComentario</i>	220 <i>operacion_relacional</i>
97 <i>Caracteres y texto</i>	160 <i>simboloFinComentario</i>	221 <i>Expresiones lógicas</i>
98 <i>Caracteres y texto</i>	161 <i>simboloSoma</i>	222 <i>condicion_basica</i>
99 <i>Caracteres y texto</i>	162 <i>simboloMultiplicacion</i>	223 <i>condiciones_and</i>
100 <i>Caracteres y texto</i>	163 <i>simboloDivision</i>	224 <i>condiciones_or</i>
101 <i>Caracteres y texto</i>	164 <i>simboloDivisionEntera</i>	225 <i>condiciones_logicas</i>
102 <i>Caracteres y texto</i>	165 <i>simboloMenor</i>	226 <i>consecucion</i>
103 <i>Caracteres y texto</i>	166 <i>simboloMayor</i>	227 <i>Expresiones y funciones</i>
104 <i>Caracteres y texto</i>	167 <i>simboloIgual</i>	228 <i>expresiones</i>
105 <i>Caracteres y texto</i>	168 <i>simboloDiferente</i>	229 <i>lista_expresiones</i>
106 <i>Caracteres y texto</i>	169 <i>simboloDiferente</i>	230 <i>llamada_funcion</i>
107 <i>Caracteres y texto</i>	170 <i>simboloDiferente</i>	231 <i>Estructuras de control</i>
108 <i>Caracteres y texto</i>	171 <i>simboloDiferente</i>	232 <i>if</i>
109 <i>Caracteres y texto</i>	172 <i>simboloDiferente</i>	233 <i>condes_if</i>
110 <i>Caracteres y texto</i>	173 <i>simboloDiferente</i>	234 <i>fin_if</i>
111 <i>Caracteres y texto</i>	174 <i>simboloDiferente</i>	235 <i>call</i>
112 <i>Caracteres y texto</i>	175 <i>simboloDiferente</i>	236 <i>condes_elif</i>
113 <i>Caracteres y texto</i>	176 <i>simboloDiferente</i>	237 <i>call</i>
114 <i>Caracteres y texto</i>	177 <i>simboloDiferente</i>	238 <i>condes_elif</i>
115 <i>Caracteres y texto</i>	178 <i>simboloDiferente</i>	239 <i>call</i>
116 <i>Caracteres y texto</i>	179 <i>simboloDiferente</i>	240 <i>call</i>
117 <i>Caracteres y texto</i>	180 <i>simboloDiferente</i>	241 <i>call</i>
118 <i>Caracteres y texto</i>	181 <i>simboloDiferente</i>	242 <i>call</i>
119 <i>Caracteres y texto</i>	182 <i>simboloDiferente</i>	243 <i>call</i>
120 <i>Caracteres y texto</i>	183 <i>simboloDiferente</i>	244 <i>call</i>
121 <i>Caracteres y texto</i>	184 <i>simboloDiferente</i>	245 <i>call</i>
122 <i>Caracteres y texto</i>	185 <i>simboloDiferente</i>	246 <i>call</i>
123 <i>Caracteres y texto</i>	186 <i>simboloDiferente</i>	247 <i>call</i>
124 <i>Caracteres y texto</i>	187 <i>simboloDiferente</i>	248 <i>call</i>
125 <i>Caracteres y texto</i>	188 <i>simboloDiferente</i>	249 <i>call</i>
126 <i>Caracteres y texto</i>	189 <i>simboloDiferente</i>	250 <i>call</i>
127 <i>Caracteres y texto</i>	190 <i>simboloDiferente</i>	251 <i>call</i>
128 <i>Caracteres y texto</i>	191 <i>simboloDiferente</i>	252 <i>call</i>
129 <i>Caracteres y texto</i>	192 <i>simboloDiferente</i>	253 <i>call</i>
130 <i>Caracteres y texto</i>	193 <i>simboloDiferente</i>	254 <i>call</i>
131 <i>Caracteres y texto</i>	194 <i>simboloDiferente</i>	255 <i>call</i>
132 <i>Caracteres y texto</i>	195 <i>simboloDiferente</i>	256 <i>call</i>
133 <i>Caracteres y texto</i>	196 <i>simboloDiferente</i>	257 <i>call</i>
134 <i>Caracteres y texto</i>	197 <i>simboloDiferente</i>	258 <i>call</i>
135 <i>Caracteres y texto</i>	198 <i>simboloDiferente</i>	259 <i>call</i>
136 <i>Caracteres y texto</i>	199 <i>simboloDiferente</i>	260 <i>call</i>
137 <i>Caracteres y texto</i>	200 <i>simboloDiferente</i>	261 <i>call</i>
138 <i>Caracteres y texto</i>	201 <i>simboloDiferente</i>	262 <i>call</i>
139 <i>Caracteres y texto</i>	202 <i>simboloDiferente</i>	263 <i>call</i>
140 <i>Caracteres y texto</i>	203 <i>simboloDiferente</i>	264 <i>call</i>
141 <i>Caracteres y texto</i>	204 <i>simboloDiferente</i>	265 <i>call</i>

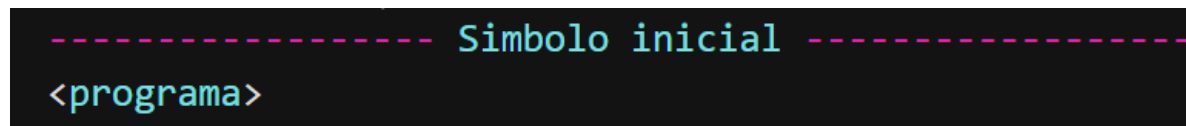
Los no terminales se organizaron por categorías temáticas —caracteres y texto, literales y tipos, delimitadores, palabras reservadas y operadores, variables, entrada/salida, arreglos, comentarios, expresiones aritméticas, relacionales y lógicas, estructuras de control,

sentencias y bloques, y parámetros y funciones— manteniendo así una correspondencia directa con la organización presentada previamente para los terminales.

Se optó por una descomposición altamente granular de los no terminales, en lugar de definir producciones más compactas. Por ejemplo, la construcción de un entero no se resuelve en un único no terminal, sino que se apoya en *<digito>*, *<digito_no_cero>*, *<ceros>* y *<lista_digitos>* por separado. Esta decisión responde a la necesidad de controlar explícitamente casos particulares, como evitar ceros a la izquierda en números enteros y eliminar ambigüedades y recursividad por la izquierda propias de una gramática pensada para análisis descendentes.

De una manera similar, las expresiones aritméticas, relacionales y lógicas se descomponen en múltiples niveles que reflejan la jerarquía de precedencia de operadores. Esto se hace por medio de una estructura de cascada utilizada de forma estándar para codificar la precedencia directamente en la gramática, evitando la dependencia de reglas externas de asociatividad.

2.3 Símbolo inicial



```
----- Símbolo inicial -----  
<programa>
```

El símbolo inicial de la gramática es *<programa>*, lo cual es coherente con la convención habitual de nombrar el símbolo inicial según la unidad más abstracta y completa que el lenguaje es capaz de generar: un programa completo, desde su primera hasta su última instrucción. Toda derivación válida en esta gramática debe partir de *<programa>* y, a través de las producciones definidas, alcanzar una cadena compuesta únicamente por terminales que constituya un programa sintácticamente correcto en el lenguaje.

2.4 Producciones

```
Documentacion >> gramatica.bnf
264 ----- Producciones -----
265 Caracteres y texto
266 <letra> ::= [a-zA-Z]
267 <digito> ::= [0-9]
268 <digito_no_cero> ::= [1-9]
269 <cero> ::= "0"
270 <lista_digitos> ::= <digito> | <digito> <lista_digitos>
271 <simbolo> ::= "@#" | "$" | "%" | "^" | "&"
272 <caracter> ::= <letra> | <digito> | <simbolo>
273 <texto> ::= <caracter> | <caracter> <texto>
274 <punto> ::= "."
275 <operadorExponencial> ::= "E" | "e"
276 <comillaDoble> ::= "\""
277 <comillaSimple> ::= "'"
278 <guionBajo> ::= "_"
279
280 Tipos y valores
281 <tipoInt> ::= "int"
282 <tipoFloat> ::= "float"
283 <tipoBool> ::= "bool"
284 <tipoChar> ::= "char"
285 <tipoString> ::= "string"
286 <tipoVoid> ::= "void"
287 <true> ::= "true"
288 <false> ::= "false"
289 <negativoNumero> ::= "-"
290 <not> ::= "!"
291 <palabraVal> ::= "val"
292 <leer> ::= "read"
293 <escribir> ::= "write"
294 <varGlobal> ::= "dg"
295 <varLocal> ::= "dl"
296 <vacio> ::= ε
297
298 Delimitadores y operadores
299 <abrir_bloque> ::= "{"
300 <cerrar_bloque> ::= "}"
301 <abrir_parentesis> ::= "("
302 <cerrar_parentesis> ::= ")"
303 <abrir_corcheteCuadrados> ::= "["
304 <cerrar_corcheteCuadrados> ::= "]"
305 <punto> ::= "."
306
307 Delimitadores y operadores
308 <abrir_parentesis> ::= "("
309 <cerrar_parentesis> ::= ")"
310 <abrir_corcheteCuadrados> ::= "["
311 <cerrar_corcheteCuadrados> ::= "]"
312 <punto> ::= "."
313 <coma> ::= ","
314 <fin_sentencia> ::= ";"
315 <asignacion> ::= "="
316 <palabraBreak> ::= "break"
317 <palabraReturn> ::= "return"
318 <palabraDefun> ::= "defun"
319 <palabraPrincipal> ::= "principal"
320 <operadorPotencia> ::= "pot"
321
322 Palabras y operadores
323 <simboloComentarioLinea> ::= "|"
324 <simboloInicioComentario> ::= "i"
325 <simboloFinComentario> ::= "f"
326 <simboloSuma> ::= "+"
327 <simboloResta> ::= "-"
328 <simboloMultiplicacion> ::= "*"
329 <simboloDivision> ::= "/"
330 <simboloDivisionEntera> ::= "//"
331 <simboloMenor> ::= "<"
332 <simboloMayor> ::= ">"
333 <simboloMenorIgual> ::= "<="
334 <simboloMayorIgual> ::= ">="
335 <simboloIgual> ::= "=="
336 <simboloDiferente> ::= "!="
337 <simboloAnd> ::= "&"
338 <simboloOr> ::= "||"
339 <palabraIf> ::= "if"
340 <palabraElif> ::= "elif"
341 <palabraElse> ::= "else"
342 <palabraWhile> ::= "while"
343 <palabraFor> ::= "for"
344 <modulo> ::= "mod"
345 <palabraDefun> ::= "defun"
346 <palabraPrincipal> ::= "main"
347 <tipoVoid> ::= "void"
348
349 Literales
350 <entero> ::= <cero> | <digito_no_cero> <lista_digitos>
351 <lista_digitos_no_cero_final> ::= <digito_no_cero> | <digito> <lista_digitos_no_cero_final>
352 <flotante> ::= <cero> <punto> <cero> | <cero> <punto> <lista_digitos_no_cero_final> | <digito_no_cero> <lista_digitos> <punto> <lista_digitos_no_cero_final>
353 <exponencial> ::= <entero> <operadorExponencial> <entero>
354 <fraccion> ::= <punto> <lista_digitos>
355 <bool> ::= <true> | <false>
356 <cadena_texto> ::= <comillaDoble> <texto> <comillaDoble>
357 <caracter_literal> ::= <comillaSimple> <caracter> <comillaSimple>
358 <valor_numerico> ::= <entero> | <flotante>
359 <valor> ::= <valor_numerico> | <exponencial> | <bool> | <caracter> | <cadena_texto> | <identificador> | <caracter_literal>
360 <tipo> ::= <tipoInt> | <tipoFloat> | <tipoBool> | <tipoChar> | <tipoString>
361 <negativo> ::= <negativoNumero> <entero> | <negativoNumero> <flotante>
```

```

Identificadores
<identificador> ::= <letra> | <letra> <resto_id>
<resto_id> ::= <letra> | <letra> <resto_id> | <dígito> <resto_id> | <guionBajo> <letra> | <guionBajo> <dígito> | <guionBajo> <letra> <resto_id> | <guionBajo> <dígito> <resto_id>

Variables
<variable> ::= <identificador>
<def_variable> ::= <palabraReservada> <tipo> <identificador> <fin_sentencia> <asignacion> <expresion> <fin_sentencia>
<def_var_global> ::= <varGlobal> <def_variable>
<def_var_local> ::= <varLocal> <def_variable>
<asignacion_variable> ::= <variable> <asignacion> <expresion> <fin_sentencia>
<variable_numerica> ::= <variable>

Entrada y salida
<lectura> ::= <clear> <variable> <fin_sentencia>
<escritura> ::= <escribir> <valor_escritura> <fin_sentencia>
<valor_escritura> ::= <entero> | <flotante> | <bool> | <cadena_texto> | <variable>

Arreglos
<indice> ::= <entero>
<arreglo> ::= <identificador>
<acceso_arreglo> ::= <arreglo> <abrir_corcheteCuerdos> <indice> <cerrar_corcheteCuerdos> ("arr j[i:1]")
<declaracion_arreglo> ::= <definicion_arreglo>
<definicion_arreglo> ::= <arreglo con tamaño> ("val int arr j[i:1] n")
<definicion_arreglo> ::= <arreglo con tamaño valores> ("val int arr j[i:1] i:2,3:7 n")
<definicion_arreglo> ::= <arreglo sin tamaño valores> ("val int arr j[i:1] i:1,2,3:7 n")
<arreglo con tamaño> ::= <palabraReservada> <tipo_arreglo> <identificador> <abrir_corcheteCuerdos> <indice> <cerrar_corcheteCuerdos> <fin_sentencia>
<arreglo sin tamaño valores> ::= <palabraReservada> <tipo_arreglo> <identificador> <abrir_corcheteCuerdos> <indice> <cerrar_corcheteCuerdos> <asignacion> <abrir_bloque> <lista_arreglo> <cerrar_bloque> <fin_sentencia>
<arreglo sin tamaño valores> ::= <palabraReservada> <tipo_arreglo> <identificador> <abrir_corcheteCuerdos> <cerrar_corcheteCuerdos> <asignacion> <abrir_bloque> <lista_arreglo> <cerrar_bloque> <fin_sentencia>
<lista_arreglo> ::= <inicioListaArreglo>
<inicioListaArreglo> ::= <valor_numerico> <continucion_lista_arreglo>
<continucion_lista_arreglo> ::= <coma> <valor_numerico> <continucion_lista_arreglo> | <vacio>
<modificacion_arreglo> ::= <acceso_arreglo> <asignacion> <valor_numerico> <fin_sentencia> ("arr j[i:1] + 8 n")
<tipo_arreglo> ::= <tipoint>
<tipo_arreglo> ::= <tipofloat>

Comentarios
<comentario_linea> ::= <simboloComentarioLinea> <texto>
<comentario_multilinea> ::= <simboloInicioComentario> <texto> <simboloFinComentario>
<comentario> ::= <comentario_linea> | <comentario_multilinea>

Operaciones aritméticas
<valor_aritmetico> ::= <valor_numerico> | <negativo> | <identificador> | <acceso_arreglo> | <llamada_funcion> | <abrir_parenthesis> <expresion_aritmetica> <cerrar_parenthesis>
<expresion_potencia> ::= <valor_aritmetico> <operadorPotencia> <expresion_potencia> | <valor_aritmetico>
<expresion_suma> ::= <expresion_suma> <operador_suma> <expresion_potencia> | <expresion_potencia>
<expresion_multiplicacion> ::= <expresion_multiplicacion> <operador_multiplicacion> <expresion_suma> | <expresion_suma>
<expresion_modulo> ::= <expresion_modulo> <operador_modulo> <expresion_multiplicacion> | <expresion_multiplicacion>
<expresion_aritmetica> ::= <expresion_modulo>
<operador_suma> ::= <simboloSuma> | <simboloResta>
<operador_multiplicacion> ::= <simboloMultiplicacion> | <simboloDivision> | <simboloDivisionEntera>

Operaciones relacionales
<operadorRelacional> ::= <simboloMenor> | <simboloMayor> | <simboloMenorIgual> | <simboloMayorIgual> | <simboloIgual> | <simboloDiferente>
<operando_relacional> ::= <abrir_corcheteCuerdos> <expresion_aritmetica> | <valor> <cerrar_corcheteCuerdos>
<operacion_relacional> ::= <operando_relacional> <operador_relacional> <operando_relacional>

Expresiones lg
<condicion_basica> ::= <abrir_parenthesis> <operacion_relacional> <cerrar_parenthesis> | <abrir_parenthesis> <bool> <cerrar_parenthesis> ("e: j[redad:] > j[18:] :?" )
<condiciones_and> ::= <condicion_basica> <simboloAnd> <condicion_basica> ("e: j[redad:] > j[18:] :? & e: j[redad:] < j[65:] :?" )
<condiciones_or> ::= <condicion_basica> <simboloOr> <condicion_basica> ("e: j[redad:] > j[18:] :? & e: j[redad:] < j[65:] :?" )
<condiciones_logicas> ::= <abrir_parenthesis> <condiciones_and> | <condiciones_or> <cerrar_parenthesis> ("e: e: j[redad:] > j[18:] :? & e: j[redad:] < j[65:] :? :?" )
<negacion> ::= <not> <condicion_basica> | <condiciones_logicas>
<expresion_logica> ::= <condiciones_and> | <condiciones_or> | <negacion>

Expresiones y funciones
<expresion> ::= <expresion_aritmetica> | <operacion_relacional> | <expresion_logica> | <valor> | <llamada_funcion>
<lista_expresiones> ::= <expresion> | <expresion> <coma> <lista_expresiones>
<llamada_funcion> ::= <identificador> <abrir_parenthesis> <lista_expresiones> <cerrar_parenthesis>

Estructuras de control
<if> ::= <palabraReservada> <abrir_parenthesis> <expresion> <cerrar_parenthesis> <abrir_bloque> <bloque> <cerrar_bloque> <demas_if>
<demas_if> ::= <cadena_elif> <fin_if> | <fin_if>
<fin_if> ::= <else> | <vacio>
<elif> ::= <palabraReservada> <abrir_parenthesis> <expresion> <cerrar_parenthesis> <abrir_bloque> <bloque> <cerrar_bloque>
<cadena_elif> ::= <elif> | <elif> <cadena_elif>
<else> ::= <palabraReservada> <abrir_bloque> <bloque> <cerrar_bloque> <fin_sentencia>
<while> ::= <palabraReservada> <abrir_parenthesis> <expresion> <cerrar_parenthesis> <abrir_bloque> <bloque> <cerrar_bloque>
<for> ::= <palabraReservada> <abrir_parenthesis> <variable> <asignacion> <valor> | <variable_numerica> <fin_sentencia> <expresion> <fin_sentencia> <incrementos> | <decrementos> <cerrar_parenthesis> <abrir_bloque> <bloque> <cerrar_bloque>
<estructura_control> ::= <if> | <while> | <for>

Sentencias y bloques
<instruccion> ::= <asignacion_variable> | <def_variable> | <def_var_global> | <def_var_local> | <estructura_control> | <comentario> | <expresion> | <break> | <return> <fin_sentencia>
<lista_instruccion> ::= <instruccion> | <instruccion> <lista_instruccion>
<bloque> ::= <lista_instruccion>
<break> ::= <palabraReservada> <fin_sentencia>
<return> ::= <palabraReservada> | <palabraReservada> <abrir_parenthesis> <expresion> | <variables> <cerrar_parenthesis> <fin_sentencia>

Parámetros y funciones
<parametro> ::= <tipo> <identificador>
<lista_parametros> ::= <vacio> | <parametro> | <parametro> <coma> <lista_parametros>
<funcion> ::= <palabraReservada> <tipo_funcion> <identificador> <abrir_parenthesis> <lista_parametros> <cerrar_parenthesis> <abrir_bloque> <bloque> <cerrar_bloque>
<lista_funciones> ::= <vacio> | <funcion> | <funcion> <lista_funciones>
<principal> ::= <tipovoid> <palabraReservada> <abrir_parenthesis> <cerrar_parenthesis> <abrir_bloque> <bloque> <cerrar_bloque> <fin_sentencia>
<programa> ::= <lista_funciones> <principal>

```

Las producciones reflejan de manera consistente la filosofía de diseño anticipada en las secciones de terminales y no terminales: descomponer cada construcción del lenguaje en unidades muy pequeñas y explícitas, en lugar de reglas compactas. Esto se aprecia especialmente en la construcción de literales, donde cada caso particular se resuelve mediante no terminales dedicados.

En las expresiones aritméticas, relacionales y lógicas se aplica la técnica estándar de precedence climbing: cada nivel de precedencia se representa como un no terminal distinto que se apoya en el nivel inmediatamente superior. Aquí conviene señalar un matiz

importante: mientras que la mayoría de la gramática evita la recursión por la izquierda (por ejemplo, en `<lista_digitos>` o `<lista_arreglo>`, que son recursivas por la derecha), las expresiones aritméticas de suma, multiplicación y módulo sí se definen con recursión por la izquierda para capturar correctamente su asociatividad. Esto es una decisión de diseño válida y común, pero implica que la gramática, tal como está, no es directamente utilizable por un analizador descendente puro sin antes eliminar esa recursión izquierda; si el analizador que se va a implementar es de este tipo, es importante mencionarlo y explicar que se contempla una transformación posterior de la gramática.

Otro patrón recurrente es el uso de `<vacío>` para modelar elementos opcionales, como listas de parámetros vacías, ausencia de bloque *else*, o funciones antes del bloque principal.

Finalmente, la producción del símbolo inicial `<programa> ::= <lista_funciones> <principal>` confirma y concreta lo comentado sobre `<programa>` donde mencionábamos que todo programa válido en el lenguaje consiste en cero o más definiciones de función seguidas obligatoriamente de un bloque principal, que actúa como punto de entrada de la ejecución.

3 Análisis de resultados

3.1 Lecciones aprendidas

Una de las principales lecciones aprendidas durante el desarrollo de la gramática fue que, antes de comenzar a crear las producciones, es necesario leer y analizar cuidadosamente el enunciado completo. A partir de esta lectura se deben identificar las restricciones, características y condiciones que debe cumplir el lenguaje. Esto permite tener claridad sobre lo que se debe implementar y evita desarrollar producciones que posteriormente tengan que modificarse por no cumplir con algún requerimiento.

También resultó útil realizar primero un boceto en papel de las diferentes partes que se necesitarían para construir la gramática. Sin embargo, durante el desarrollo se comprendió que no es necesario definir desde el inicio absolutamente todos los elementos que se utilizarán. Conforme se van creando las producciones pueden surgir nuevas necesidades, por lo que es posible identificar y agregar posteriormente los no terminales o producciones necesarias. Esto permitió que el desarrollo fuera más flexible y se adaptara a las necesidades que iban apareciendo.

Otra experiencia importante fue utilizar ejemplos de cómo se representaría el lenguaje directamente en código. Contar con ejemplos concretos permitió comprobar cómo deberían estructurarse las producciones y verificar si la gramática estaba tomando la forma esperada. Esto facilitó identificar si las reglas creadas realmente permitían representar las estructuras planteadas en los requerimientos.

También se aprendió que no es conveniente crear producciones con nombres demasiado específicos cuando estas podrían necesitar utilizarse en diferentes partes de la gramática. Los nombres de los no terminales deben ser claros, generales y representativos de su función, de manera que puedan reutilizarse cuando sea necesario. Además, utilizar nombres descriptivos facilita que otra persona pueda comprender qué representa cada producción y cómo se relaciona con las demás.

Durante el desarrollo se presentaron dificultades relacionadas con la semántica y el manejo de los tipos de datos. Inicialmente se dedicó bastante atención a determinar cómo controlar estas restricciones dentro de la gramática. Posteriormente, mediante la aclaración realizada con el profesor, se comprendió mejor cuáles aspectos correspondían directamente a la sintaxis de la gramática y cuáles podían ser tratados mediante validaciones semánticas. Esto permitió enfocar el desarrollo en las características que realmente debían ser representadas mediante las producciones.

Otra práctica que resultó útil fue utilizar nombres como subtítulos o agrupaciones para organizar las producciones. Se agruparon los elementos relacionados, como variables, arreglos, expresiones, operadores, estructuras de control y funciones. Esto permitió mantener el archivo más ordenado, facilitar la lectura y encontrar con mayor facilidad las producciones necesarias durante el desarrollo.

Otra lección aprendida fue que no todas las características deben restringirse más allá de lo indicado en el enunciado. Por ejemplo, al trabajar con la sentencia `break`, se comprendió que no era necesario agregar restricciones adicionales sobre dónde podía utilizarse, ya que el requerimiento únicamente solicita permitir esta sentencia. Esto ayudó a evitar agregar reglas innecesarias que podrían limitar el comportamiento definido para el lenguaje.

Por último, se aprendió la importancia de trabajar en conjunto cuando la tarea es desarrollada en pareja. Trabajar con el compañero permitió que ambos conocieran las decisiones tomadas, comprendieran cómo se estaba construyendo la gramática y pudieran

discutir las diferentes alternativas antes de implementarlas. Esto facilitó la detección de errores, la distribución del trabajo y la toma de decisiones de manera conjunta, evitando que una sola persona desconociera partes importantes del desarrollo.

3.2 Objetivos alcanzados

- ✓ Desarrollar una gramática BNF para un lenguaje imperativo orientado a sistemas empotrados.
- ✓ Definir la estructura general del lenguaje mediante terminales, no terminales, símbolo inicial y producciones.
- ✓ Implementar el manejo de variables, tipos de datos y arreglos.
- ✓ Implementar expresiones aritméticas, relacionales y lógicas, respetando las reglas de precedencia establecidas.
- ✓ Incorporar estructuras de control, funciones, parámetros y valores de retorno.
- ✓ Incorporar las operaciones de entrada y salida mediante read y write.
- ✓ Implementar el procedimiento principal y los mecanismos de asignación.
- ✓ Incorporar comentarios y los delimitadores establecidos para el lenguaje.
- ✓ Organizar las producciones de manera estructurada, clara y específica, evitando una producción genérica para todas las expresiones.

3.3 Objetivos no alcanzados o pendientes de validación

- ✗ Completar la validación semántica de los tipos de datos y su compatibilidad en las diferentes operaciones.
- ✗ Validar completamente las restricciones de tipos en expresiones aritméticas, relacionales y lógicas.
- ✗ Comprobar la compatibilidad entre los parámetros, argumentos y valores de retorno de las funciones.
- ✗ Validar completamente las restricciones de read y write.
- ✗ Realizar pruebas exhaustivas para comprobar que las estructuras válidas sean aceptadas y las inválidas rechazadas.
- ✗ Verificar completamente la precedencia y asociatividad de todas las operaciones.

3.4 Bitácora

<https://github.com/HeldyisAE/TA01-Compiladores-e-Interpretes-gramatica.git>